

Práctica 4: Módulos Odoo Modelo y Vista.

Andrea Sofía Pais Dos Santos

February 1, 2026

1 Índice

2. Introducción	página 3
3. Actividad 01 - Liga Fútbol	página 3
• Modificar lógica de puntuación de partidos	
• Botones para alterar goles de los partidos	
• Web controller para eliminar empates	
• Informe para cada partido	
• Wizard para crear nuevos partidos	
• Vista Graph	
• Errores encontrados	
4. Actividad 02 - Pruebas Api Rest Socios	página 15
• Probar crear, modificar, consultar y eliminar	
• Utilizar una herramienta como: (Postman, Thunder Client)	
• Errores Encontrados	
5. Actividad 03 - Bot de Telegram conectado a la Api Rest	página 16
• Crear bot de Telegram configurando con BotFather	
• Escuchar órdenes enviados por los usuarios	
• El bot tiene que soportar el bot	
• Errores encontrados	
6. Actividad 04 - Generar de imágenes aleatorias con Web Controller	página 21
• Crear un Web Controller	
• Errores encontrados	
7. Conclusiones	página 23

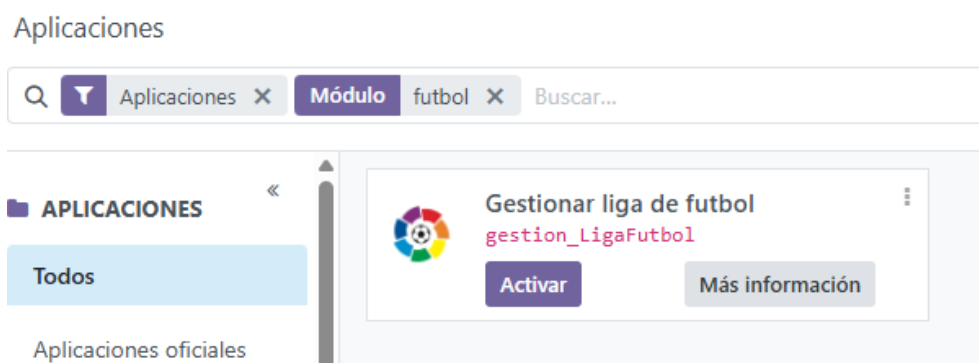
2 Introducción

El objetivo de la práctica es aplicar lo aprendido sobre informes, vistas, wizard y controladores web para seguir adquiriendo conceptos y ser capaces de crear módulos por nuestra cuenta cada vez más complejos. Nos vamos a basar en ejemplos del 01 al 06 del repositorio que nos proporcionan en el enunciado: [repositorio de ejemplo](#).

Hay que tener en cuenta que para ir viendo si funciona los módulos y sus respectivas vistas estén correctos tenemos que levantar el contenedor en docker, iniciar en Odoo con nuestros datos, actualizar la lista de aplicaciones y realizar lo que necesitemos: activar módulo, actualizar módulo, etc.

3 Actividad 01 - Liga Fútbol

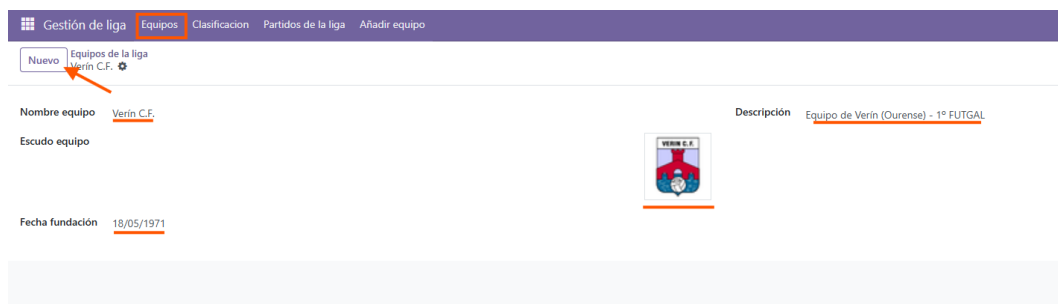
Para esta actividad importamos el módulo de base que está en el repositorio que se ha dado para la práctica: [repositorio de ejemplo](#). Teniendo esta base, levantados el contenedor de docker que teníamos para las anteriores prácticas y iniciamos en Odoo con nuestros datos. Si estamos en modo desarrollador, actualizamos nuestra lista de aplicaciones e instalamos el módulo nuevo "Liga Fútbol".



3.1 Modificar lógica de puntuación de partidos

Lo que hay que añadir en este apartado es que si en un partido hay más de 4 goles de diferencia el que gana se le suma 4 puntos y el que pierde -1 punto. Si esa diferencia no existe y no hay un empate, el equipo ganador se sigue llevando 3 puntos y el perdedor 0 puntos. En caso de empate los equipos se llevan 1 punto cada uno.

Para saber si nos va a funcionar el cambio en la lógica, cree dos equipos de fútbol inspirados en la liga de Ourense de 1^o FutGal.



Al tener los 2 equipos, en clasificación vamos a tener todos los datos de los resultados de los partidos y los puntos que tenemos que modificar ahora, porque de momento solo están los puntos de victoria de 3 puntos, de empate 1 punto y de derrota 0 punto.

	Puntos	Jugados	Goles A F...	Goles En ...	Victorias	Empates	Derrotas
Verín C.F.	0	0	0	0	0	0	0
C.F. Monterrei	0	0	0	0	0	0	0

Para realizar los cambios que tenemos que hacer, hay que cambiar el archivo models "liga partido". En el documento ya hay una base de como funciona los puntos base.

```

1 def actualizoRegistrosEquipo(self):
2     #Recorremos partidos y equipos
3     for recordEquipo in self.env['liga.equipo'].search([]):
4         #Como recalculamos todo, ponemos de cada equipo todo a cero
5         recordEquipo.victorias=0
6         recordEquipo.empates=0
7         recordEquipo.derrotas=0
8         recordEquipo.goles_a_favor=0
9         recordEquipo.goles_en_contra=0
10        recordEquipo.puntos= 0
11
12        for recordPartido in self.env['liga.partido'].search([]):
13
14            #Si es el equipo de CASA
15            if recordPartido.equipo_casa.nombre == recordEquipo.nombre:
16
17                #Miramos si es victoria o derrota
18                if recordPartido.goles_casa > recordPartido.goles_fuera:
19                    recordEquipo.victorias=recordEquipo.victorias + 1
20                    # Si los goles marcados por el equipo de casa es mas de 4
21                    # goles por encima del equipo visitante
22                    if recordPartido.goles_casa - recordPartido.goles_fuera >= 4:
23                        #Se suman los 4 puntos
24                        recordEquipo.puntos = recordEquipo.puntos + 4
25                    else:
26                        #Si no hay esa diferencia se le suma al equipo de casa 3
27                        # puntos por victoria
28                        recordEquipo.puntos = recordEquipo.puntos + 3
29                elif recordPartido.goles_casa < recordPartido.goles_fuera:
30                    recordEquipo.derrotas = recordEquipo.derrotas + 1
31                    #Si los goles marcados por el equipo de fuera es mas de 4
32                    # goles que el otro equipo, al equipo de casa
33                    if recordPartido.goles_fuera - recordPartido.goles_casa >= 4:
34                        #Se le resta 1 punto
35                        recordEquipo.puntos = recordEquipo.puntos - 1
36                    else:
37                        recordEquipo.empates = recordEquipo.empates + 1
38                        #Un punto por empate
39                        recordEquipo.puntos = recordEquipo.puntos + 1

```

```

38         #Sumamos goles a favor y en contra
39         recordEquipo.goles_a_favor = recordEquipo.goles_a_favor+
            recordPartido.goles_casa
40         recordEquipo.goles_en_contra = recordEquipo.goles_en_contra+
            recordPartido.goles_fuera
41
42         #Si es el equipo de FUERA
43         if recordPartido.equipo_fuera.nombre==recordEquipo.nombre:
44
45             #Miramos si es victoria o derrota
46             if recordPartido.goles_casa < recordPartido.goles_fuera:
47                 recordEquipo.victorias = recordEquipo.victorias + 1
48                 # Si los goles marcados por el equipo de fuera es mas de 4
                    goles por encima del equipo en casa
49                 if recordPartido.goles_fuera - recordPartido.goles_casa >= 4:
50                     #Se suman los 4 puntos
51                     recordEquipo.puntos = recordEquipo.puntos + 4
52                 else:
53                     #Si no hay esa diferencia se le suma al equipo de casa 3
                        puntos por victoria
54                     recordEquipo.puntos = recordEquipo.puntos + 3
55                 elif recordPartido.goles_casa > recordPartido.goles_fuera:
56                     recordEquipo.derrotas = recordEquipo.derrotas + 1
57                     # Si los goles marcados por el equipo de casa es mas de 4
                        goles por encima del equipo de fuera
58                     if recordPartido.goles_casa - recordPartido.goles_fuera >= 4:
59                         # Se le resta 1 punto
60                         recordEquipo.puntos = recordEquipo.puntos - 1
61                 else:
62                     recordEquipo.empates = recordEquipo.empates + 1
63                     #Uno por empate
64                     recordEquipo.puntos = recordEquipo.puntos + 1
65
66             #Sumamos goles a favor y en contra
67             recordEquipo.goles_a_favor = recordEquipo.goles_a_favor +
                recordPartido.goles_fuera
68             recordEquipo.goles_en_contra = recordEquipo.goles_en_contra +
                recordPartido.goles_casa

```

Como base ya viene una estructura que añade 3 puntos por victoria y los puntos por empate y partido perdido. Lo que nos pide es añadir al equipo ganador sea de casa o visitante y los goles son superior a 4 goles del rival hay que sumarle +4 puntos al equipo ganador.

3.2 Botones para alterar goles de los partidos

El objetivo de este apartado es crear dos botones que sumen goles de 2 en 2 por un lado a los equipos locales y también a los visitantes. Luego se tiene que recalcular la clasificación.

Lo primero es en el archivo "liga partido.py" añadimos las funciones que harán que cambien los goles y aumenten según que equipo sea.

```

1      # Sumar los 2 goles a los equipos locales
2      def sumar_goles_locales(self):
3          #Buscar todos los partidos
4          partidos = self.search([])
5          for partido in partidos:
6              partido.goles_casa += 2
7          self.actualizoRegistrosEquipo()
8          return True
9
10     # Sumar los 2 goles a los equipos visitantes
11     def sumar_goles_visitantes(self):
12         #Buscar todos los partidos
13         partidos = self.search([])
14         for partido in partidos:
15             partido.goles_fuera += 2
16         self.actualizoRegistrosEquipo()
17         return True

```

Lo complicado era crear los botones en la vista, porque en la vista Kanban los botones se añadían dentro de la tarjeta y tenía que modificar todos los equipos. La manera de solucionarlo para que los botones aparezca arriba en ambas vistas es meterlo dentro de la etiqueta "header" que en lista no funciona a no ser que se le añada display="always".

```

1      <!-- Vista List -->
2      <record id="liga_partido_view_list_nueva" model="ir.ui.view">
3          <field name="name">Lista de partidos de la liga</field>
4          <field name="model">liga.partido</field>
5          <field name="arch" type="xml">
6              <list>
7                  <header>
8                      <button name="sumar_goles_locales" string="+2 Goles Locales" type=
9                          "object" display="always"/>
10                     <button name="sumar_goles_visitantes" string="+2 Goles Visitantes"
11                         type="object" display="always"/>
12                 </header>
13                 <!-- Indicamos que atributos usaremos al hacer la vista List -->
14                 <field name="equipo_casa" />
15                 <field name="goles_casa" />
16                 <field name="equipo_fuera" />
17                 <field name="goles_fuera" />
18             </list>
19         </field>
20     </record>
21     <!-- Y lo mismo en la vista kanban -->
22     <record id="liga_partido_view_kanban" model="ir.ui.view">
23         <field name="name">Lista de partidos de la liga</field>
24         <field name="model">liga.partido</field>
25         <field name="arch" type="xml">
26             <!-- Agrupamos por el atributo "parent_id"-->
27             <kanban>
28                 <!-- Indicamos que atributos usaremos al hacer la vista Kanban -->
29                 <header>
30                     <button name="sumar_goles_locales" string="+2 Goles Locales" type=
31                         "object" display="always"/>
32                     <button name="sumar_goles_visitantes" string="+2 Goles Visitantes"
33                         type="object" display="always"/>

```

```

30     </header>
31     <field name="equipo_casa" />
32     <field name="goles_casa" />
33     <field name="equipo_fuera" />
34     <field name="goles_fuera" />
35     ...

```

Por lo que visualmente quedaría así los botones de sumar goles:

Equipo local	Goles Casa	Equipo visitante	Goles Fu...
☐ C.F. Monterrei	3	Verín C.F.	8
☐ C.F. Monterrei	0	Verín C.F.	3

Así quedaría si sumamos 2 goles a los equipos locales y dos a los visitantes.

Equipo local	Goles Casa	Equipo visitante	Goles Fu...
☐ C.F. Monterrei	5	Verín C.F.	2
☐ C.F. Monterrei	0	Verín C.F.	3

3.3 Web controller para eliminar empates

Para crear un Web Controller vamos a necesitar crear en el módulo un carpeta "controllers" que necesita un archivo "init" y un archivo "main". Como cogimos como base el que está en el repositorio ya está creado. Ahora lo que vamos es a editar el archivo "main".

```

1  #Al acceder a http://localhost:8069/eliminarempates borra los partidos empatados
2  @http.route("/eliminarempates",type="http",auth="public", website=True)
3  def eliminar_empates(self,**kw):
4      #Buscamos los partidos
5      partidos = request.env["liga.partido"].sudo().search([])
6      #Empezamos un contador
7      contador = 0
8      #Buscamos que partidos tiene los goles iguales
9      for partido in partidos:
10         if partido.goles_casa == partido.goles_fuera:
11             #Borrar los registros
12             partido.unlink()
13             #Vamos sumando los partidos empatados borrados
14             contador += 1
15         request.env["liga.partido"].sudo().actualizoRegistrosEquipo()
16         #Nos devuelve unos textos que nos dice cuantos partidos se borrar con empates
17         return "<h1>Operacion realizada</h1><p>Se han borrado un total de partidos con
            empate: %s</p>" % contador

```

Ahora para probarlo actualizamos el módulo, creamos un partido con goles empatados y entramos en la url `http://localhost:8069/eliminarempates`.

Gestión de liga Equipos Clasificación Partidos de la liga Añadir equipo My Company A					
Nuevo Partidos de la liga ⚙		Q Buscar...		1-2 / 2 < > ☰ 🖨	
☐		Equipo local	Goles Casa	Equipo visitante	Goles Fu...
☐	+2 Goles Locales +2 Goles Visitantes	C.F. Monterrei	3	Verín C.F.	8
☐	+2 Goles Locales +2 Goles Visitantes	C.F. Monterrei	2	Verín C.F.	2

Al entrar en la ruta vemos que se ha eliminado el partido empatado.



Para ver que se borró volvemos a la lista de partidos y vemos que ya se ha borrado el partido.

Gestión de liga Equipos Clasificación Partidos de la liga Añadir equipo My Company A					
Nuevo Partidos de la liga ⚙		Q Buscar...		1-1 / 1 < > ☰ 🖨	
☐		Equipo local	Goles Casa	Equipo visitante	Goles Fu...
☐	+2 Goles Locales +2 Goles Visitantes	C.F. Monterrei	3	Verín C.F.	8
➡					

3.4 Informe para cada partido

Para generar un informe de cada partido hay que crear un botón para imprimir el informe y nos lo genere y una plantilla de como se verá visualmente el informe. En vistas creo una nueva vista "liga partido informe" y la diseñamos con todos los datos principales que necesitan un informe.

```

1      <?xml version="1.0" encoding="utf-8"?>
2  <odoo>
3      <!--Tienen que estar los atributos en ingles-->

```



```

4 <record id="action_report_partido" model="ir.actions.report">
5   <field name="name">Resultado del Partido</field>
6   <field name="model">liga.partido</field>
7   <field name="report_type">qweb-pdf</field>
8   <field name="report_name">gestion_LigaFutbol.informe_partido_plantilla</field>
9   <field name="print_report_name">'Partido -- %s' % (object.equipo_casa.nombre)
10     </field>
11   <field name="binding_model_id" ref="model_liga_partido"/>
12   <field name="binding_type">report</field>
13 </record>
14
15 <template id="informe_partido_plantilla">
16   <t t-call="web.html_container">
17     <t t-foreach="docs" t-as="o">
18       <t t-call="web.external_layout">
19         <div class="datos">
20           <!--Datos de los equipos -->
21           <div class="datos2">
22             <p>EQUIPO LOCAL</p>
23             <p t-field="o.equipo_casa.nombre"></p>
24           </div>
25           <div class="datos2">
26             <p>EQUIPO VISITANTE</p>
27             <p t-field="o.equipo_fuera.nombre"></p>
28           </div>
29         </div>
30         <!--resultado del partido con goles-->
31         <div class="pagina">
32           <div class="text-center" style="border-bottom: 2px solid black
33             ; margin-bottom: 20px;">
34             <h2>ACTA DEL PARTIDO</h2>
35           </div>
36
37           <div class="row align-items-center justify-content-center"
38             style="font-size: 20px; margin-top: 50px;">
39             <div class="col-5 text-end"> <strong t-field="o.
40               equipo_casa.nombre"/>
41             </div>
42
43             <div class="col-2 text-center" style="background-color: #
44               f2f2f2; border-radius: 10px; width: 100px; padding: 10
45               px; margin-left: 5px; margin-right: 5px;">
46               <strong>
47                 <span t-field="o.goles_casa"/> - <span t-field="o.
48                   goles_fuera"/>
49               </strong>
50             </div>
51
52             <div class="col-5 text-start"> <strong t-field="o.
53               equipo_fuera.nombre"/>
54             </div>
55           </div>
56           <!--Pie de acta que sino quedaba muy vacio-->
57           <div class="text-center" style="margin-top: 100px; color: grey
58             ;">
59             <p>Informe generado por el Sistema de Gestion de la Liga</
60               p>
61           </div>

```

```

52         </div>
53     </t>
54 </t>
55 </t>
56 </template>
57 </odoo>

```

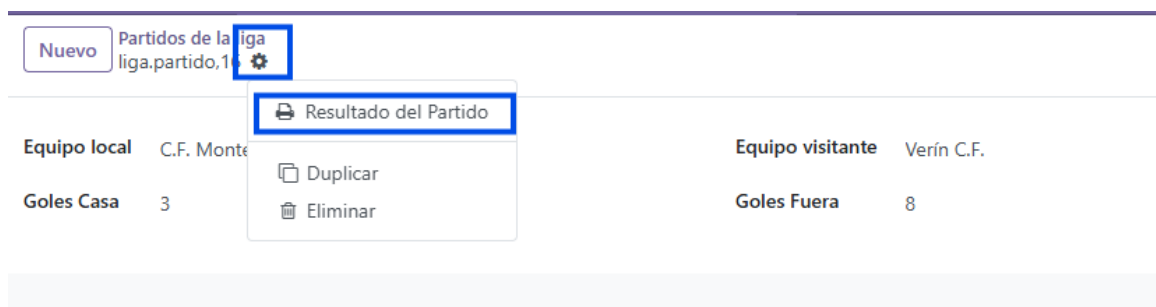
Además hay que añadir al archivo manifest principal del módulo en "data" la nueva vista creada:

```

1  'data': [
2      #'security/groups.xml',
3      'security/ir.model.access.csv',
4
5      #Aqui distintas vistas de equipo (vistas diferentes, mismo modelo)
6      'views/liga_equipo.xml',
7      'views/liga_equipo_clasificacion.xml',
8      #Vista a un informe
9      'report/liga_equipo_clasificacion_report.xml',
10     #Aqui vista sobre los partidos
11     'views/liga_partido.xml',
12     'wizard/liga_equipo_wizard.xml',
13     # Poner la nueva vista
14     'views/liga_partido_informe.xml'
15 ],
16

```

Tras diseñar la plantilla del informe para probarlo hay que actualizar el módulo y en el apartado de partido, seleccionamos el partido del que necesitamos el informe y en el icono de ajustes aparece un apartado que es "Resultado del Partido".



Genera un pdf que se abre automáticamente con los datos del partido.

**FUNDACIÓN
LALIGA**
 GesLiga
 España

EQUIPO LOCAL
 C.F. Monterrei
 EQUIPO VISITANTE
 Verín C.F.

ACTA DEL PARTIDO

C.F. Monterrei
 3 - 8
 Verín C.F.

Informe generado por el Sistema de Gestión de la Liga

Página 1 / 1

3.5 Wizard para crear nuevos partidos

El objetivo de este apartado es crear un partido mediante un formulario emergente que guarda datos temporales. Para crear un Wizard hay que generar un modelo nuevo "partido wizard".

```

1  # -*- coding: utf-8 -*-
2  from odoo import models, fields
3
4  class PartidoWizard(models.TransientModel):
5      #Nombre de la tabla temporal que se va a crear
6      _name = 'liga.partido.wizard'
7      _description = 'Wizard para crear partidos'
8      #Campos del formulario
9      equipo_casa = fields.Many2one('liga.equipo', string="Equipo Local", required=
        True)
10     equipo_fuera = fields.Many2one('liga.equipo', string="Equipo Visitante",
        required=True)
11     goles_casa = fields.Integer(string="Goles Local", default=0)

```

```

12     goles_fuera = fields.Integer(string="Goles Visitante", default=0)
13
14     def crear_partido(self):
15         # Esta funcion crea el registro permanente en el modelo liga.partido
16         self.env['liga.partido'].create({
17             'equipo_casa': self.equipo_casa.id,
18             'equipo_fuera': self.equipo_fuera.id,
19             'goles_casa': self.goles_casa,
20             'goles_fuera': self.goles_fuera,
21         })
22         # Recalcular la clasificacion tras poner el partido nuevo
23         self.env['liga.partido'].actualizarRegistrosEquipo()
24         return {'type': 'ir.actions.act_window_close'} # Cierra la ventana
                emergente

```

Ahora hay que hacer una vista para que se genere el formulario anterior "partido wizard vista"

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3     <record id="partido_wizard_vista" model="ir.ui.view">
4         <field name="name">liga.partido.wizard.form</field>
5         <field name="model">liga.partido.wizard</field>
6         <field name="arch" type="xml">
7             <form string="Crear Nuevo Partido">
8                 <group>
9                     <group>
10                         <field name="equipo_casa"/>
11                         <field name="goles_casa"/>
12                     </group>
13                     <group>
14                         <field name="equipo_fuera"/>
15                         <field name="goles_fuera"/>
16                     </group>
17                 </group>
18                 <group>
19                     <button name="crear_partido" string="Generar Partido" type="object" />
20                     <button string="Cancelar" special="cancel"/>
21                 </group>
22             </form>
23         </field>
24     </record>
25
26     <record id="action_partido_wizard" model="ir.actions.act_window">
27         <field name="name">Anadir Partido</field>
28         <field name="res_model">liga.partido.wizard</field>
29         <field name="view_mode">form</field>
30         <field name="target">new</field>
31     </record>
32
33     <menuitem id="menu_partido_wizard"
34         name="Wizard Crear Partido"
35         parent="liga_base_menu"
36         action="action_partido_wizard"
37         sequence="50"/>
38 </odoo>

```

Hay que tener en cuenta que al crear estos archivos ahora en "init" y "manifest" hay que añadir esto nuevo:

En el archivo "init" de la carpeta vista:

```
1 from . import partido_wizard
```

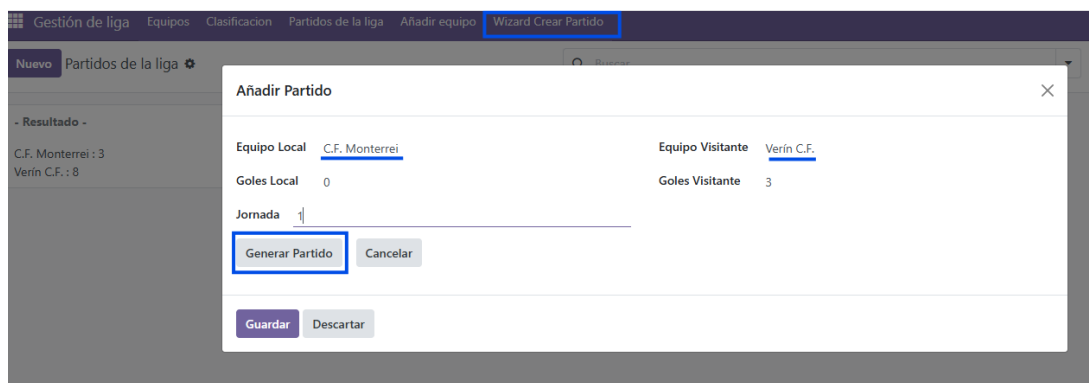
En el archivo "manifest" en data:

```
1 'views/partido_wizard_vista.xml'
```

Un error que tuve fue que no me cargaba la vista y tenía que añadir los permisos al archivo "ir.model.access.csv"

```
1 acl_partido_wizard_vista,liga.equipo_wizard,model_partido_wizard,,1,1,1,1
```

Aquí vemos que nos sale el apartado nuevo arriba en el menú que al pinchar nos abre una ventana emergente para añadir los datos del partido de forma rápida.



Y ya dentro de los partidos nos sale el que he creado.

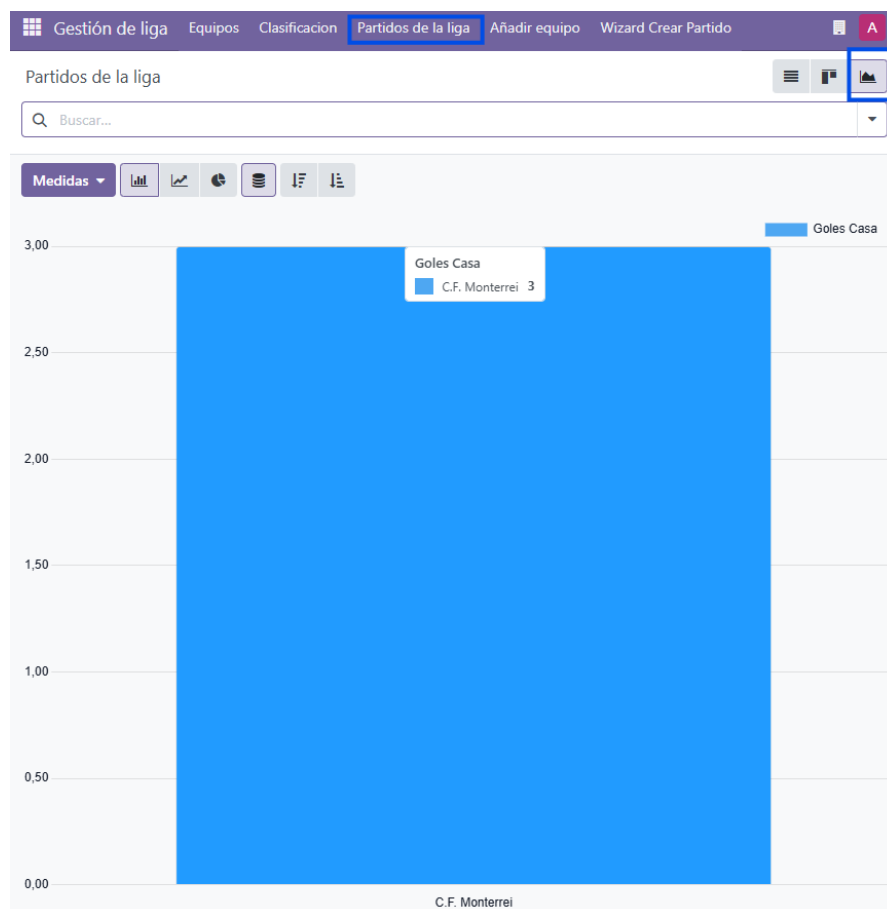


3.6 Vista Graph

Para crear una gráfica me basé en la gráfica que estaba en el apartado de equipos y cambié los valores a goles y equipos locales. Además hay que añadir el parámetro "graph".

```
1  ...
2  <field name="view_mode">list,kanban,form,graph</field>
3  ...
4  <!-- Vista Graph que muestra informacion de los goles locales en total-->
5  <record model="ir.ui.view" id="liga_partido_view_graph">
6    <field name="name">Goles de los equipos locales</field>
7    <field name="model">liga.partido</field>
8    <field name="type">graph</field>
9    <field name="arch" type="xml">
10      <!-- Cada fila es un equipo y la altura los goles -->
11      <graph string="Puntos por equipo">
12        <field name="equipo_casa" group="True" type="row"/>
13        <field name="goles_casa" group="True" type="measure"/>
14      </graph>
15    </field>
16  </record>
```

Ahora para ver la gráfica actualizamos el módulo y dentro del apartado de partidos vemos una nueva vista. Y se ve una gráfica de los equipos locales con los goles totales.



3.7 Errores encontrados

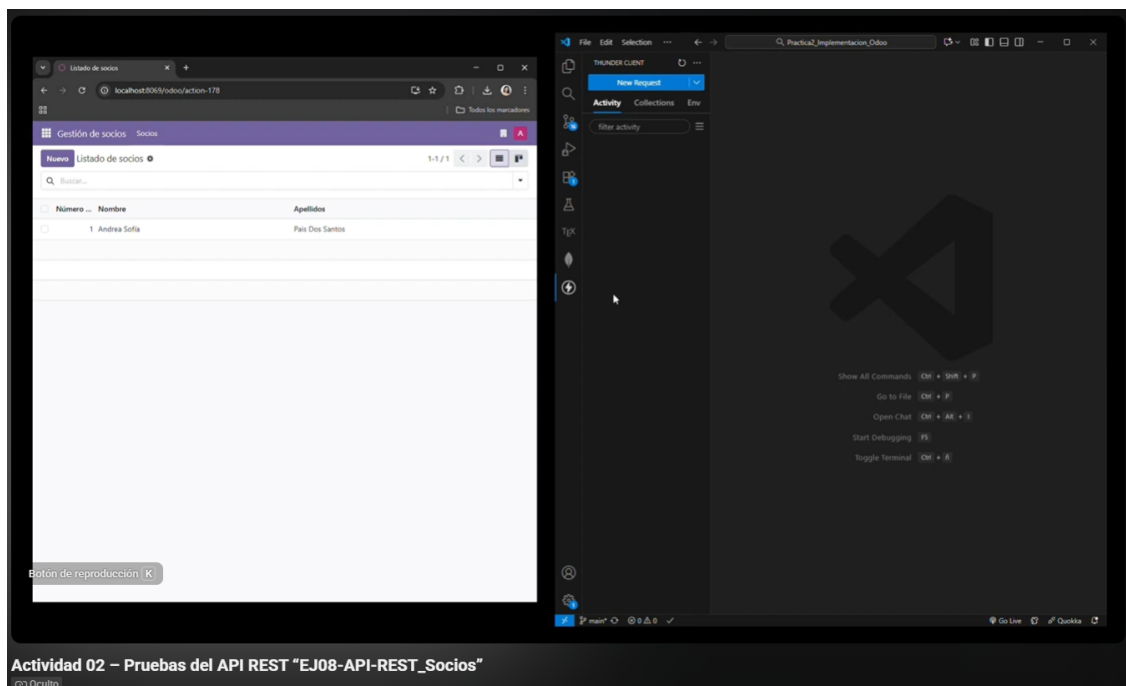
El primer error que se tuvo en esta actividad fue al intentar abrir el módulo nos daba un error que ya tuvimos en otras prácticas que era la utilización de la etiqueta `<tree>` que ya no se utiliza y hay que cambiarlo por `<list>`. se cambió en todos los archivos de las vistas y el módulo ya se puede abrir correctamente.

Otro error que sucede mucho es equivocarme al llamar con el nombre correcto funciones que creo por ejemplo me dio error en el controlador porque había llamado mal a la función de recargar.

4 Actividad 02 - Pruebas Api Rest Socios

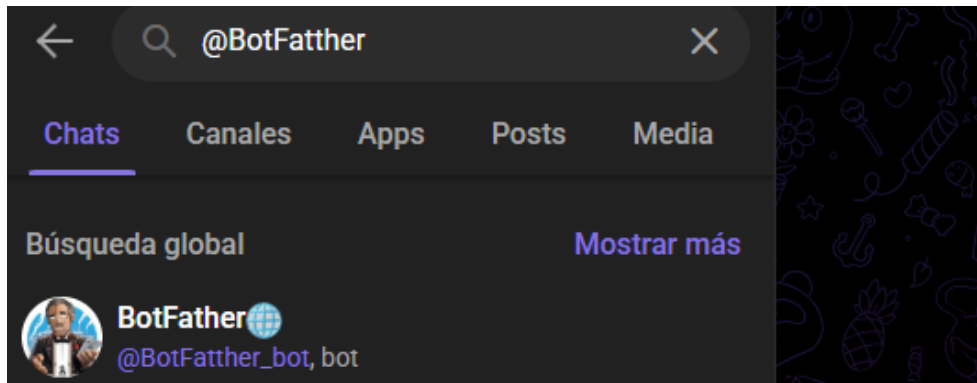
La actividad 2 consiste en realizar las 4 operaciones básicas (crear, modificar, listar y eliminar) usando en mi caso Thunder Client y ser capaz de ver los resultados en Odoo. La pruebas realizadas se encuentran en este vídeo de youtube [pinchar aquí](#).

Si el enlace no funciona, el video está adjunto en el repositorio del github.

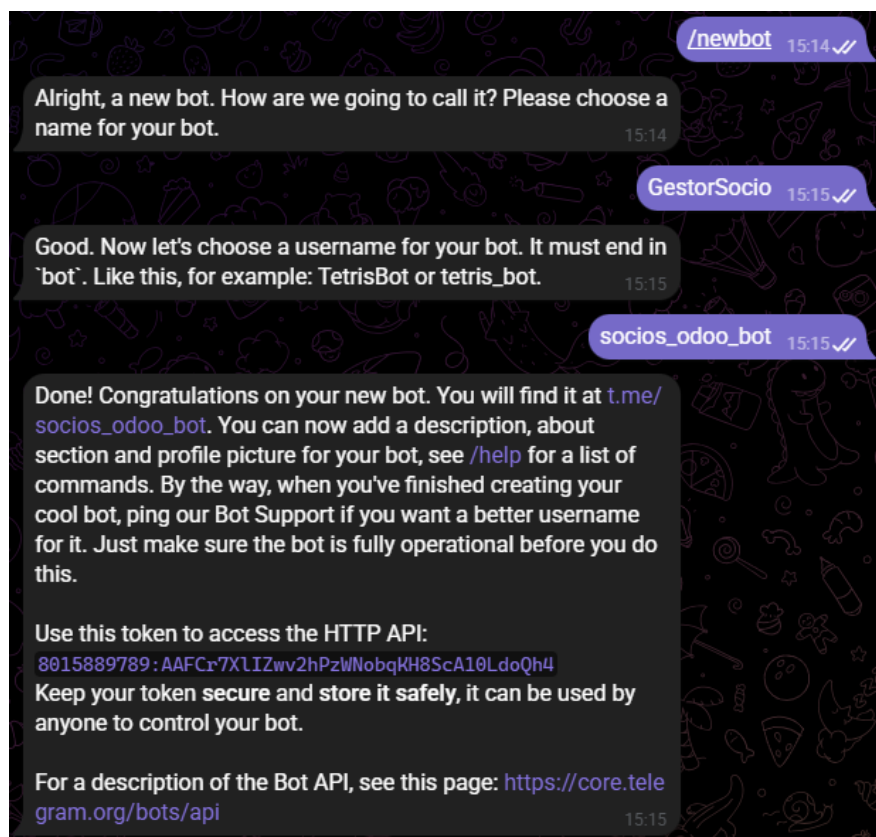


5 Actividad 03 - Bot de Telegram conectado a la Api Rest

En esta actividad vamos a crear primero un bot de Telegram que se va a configurar con BotFather.



Ahora vamos a crear un bot para esta actividad, poniendo el comando `/newbot`. Y le damos un nombre al bot y al usuario que tiene que acabar en "bot". El BotFather nos da un Token que lo vamos a tener que usar en el script de Python.



Tras tener el bot tenemos que instalar las herramientas que permitan hablar al bot con Telegram y con Odoo. Abrimos una terminal e instalamos las dependencias necesarias:

```
"python -m pip install python-telegram-bot requests"
```



```

PS C:\WINDOWS\system32> python -m pip install python-telegram-bot requests
Collecting python-telegram-bot
  Downloading python_telegram_bot-22.6-py3-none-any.whl.metadata (17 kB)
Collecting requests
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting httpcore>=1.0.9 (from python-telegram-bot)
  Downloading httpcore-1.0.9-py3-none-any.whl.metadata (21 kB)
Collecting httpx<0.29,>=0.27 (from python-telegram-bot)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting anyio (from httpx<0.29,>=0.27->python-telegram-bot)
  Downloading anyio-4.12.1-py3-none-any.whl.metadata (4.3 kB)
Collecting certifi (from httpx<0.29,>=0.27->python-telegram-bot)
  Downloading certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Collecting idna (from httpx<0.29,>=0.27->python-telegram-bot)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting h11>=0.16 (from httpcore>=1.0.9->python-telegram-bot)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl.metadata (38 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Downloading python_telegram_bot-22.6-py3-none-any.whl (737 kB)
----- 737.3/737.3 kB 10.8 MB/s 0:00:00
Downloading httpx-0.28.1-py3-none-any.whl (73 kB)
Downloading httpcore-1.0.9-py3-none-any.whl (78 kB)

```

Ahora hay que crear un archivo de python fuera de las carpetas de los módulos porque el Bot de Telegram es una aplicación independiente, este bot le va a pedir cosas a Odoo y si está dentro de algún módulo, Odoo intentará cargarlo como si fuera parte del sistema y dará errores.

En el archivo tenemos que escribir esto, porque el ejercicio nos pide que el bot soporte cuatro órdenes básicas (crear, modificar, consultar y eliminar) y si no es ninguna que nos devuelva una respuesta de orden no soportada.

```

1  import requests
2  from telegram import Update
3  from telegram.ext import ApplicationBuilder, MessageHandler, filters
4
5  # configuracion
6  TOKEN = '8015889789:AAFcr7XlIZwv2hPzWNobqKH8ScA10LdoQh4'
7  URL = 'http://localhost:8069/gestion/apiREST/socio'
8
9  async def manejar_mensaje(update: Update, context):
10     # Cogemos el texto y lo separamos por comas (la orden, nombre, apellidos, numero)
11     texto = update.message.text
12     partes = [parte.strip() for parte in texto.split(',')]
13     orden = partes[0] # esta es la orden
14
15     try:
16         # si la orden es igual a crear
17         if orden == "Crear":
18             # pasamos los datos (nombre, apellidos, numero socio)
19             datos = {"nombre": partes[1], "apellidos": partes[2], "num_socio": partes
20                     [3]}
21             # Envia la informacion a la url
22             r = requests.post(URL, json=datos)
23             # Nos devuelve el contenido del mensaje, el f"" se usa para crear frase
24             # combinando texto normal con variables
25             respuesta = f"Socio creado: {r.text}"
26
27         # si la orden es igual a consultar
28         elif orden == "Consultar":

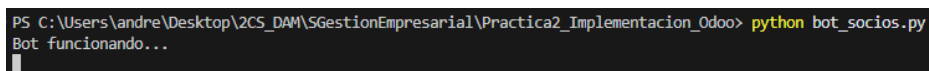
```

```

27         # los datos ( orden y el numero de socio)
28         params = {"data": '{"num_socio": "' + partes[1] + '"}'}
29         # Envia la informacion a la url
30         r = requests.get(URL, params=params)
31         # Nos devuelve el contenido del mensaje
32         respuesta = f"Datos: {r.text}"
33
34     # si la orden es igual a borrar
35     elif orden == "Borrar":
36         # datos (orden y numero de socio)
37         params = {"data": '{"num_socio": "' + partes[1] + '"}'}
38         # Envia la informacion a la url
39         r = requests.delete(URL, params=params)
40         # Nos devuelve el contenido del mensaje
41         respuesta = f"Borrado: {r.text}"
42     # si la orden es igual a modificar
43     elif orden == "Modificar":
44         # datos(orden,nombre,apellidos,numero de socio)
45         datos = {"nombre": partes[1], "apellidos": partes[2], "num_socio": partes
46                 [3]}
47         # Envia la informacion a la url
48         r = requests.put(URL, json=datos)
49         # Nos devuelve el contenido del mensaje
50         respuesta = f"Socio modificado: {r.text}"
51     else:
52         # Si no es ninguna de las ordenes devuelve el mensaje
53         respuesta = "Orden no soportada"
54
55     except Exception as error:
56         respuesta = f"Error, igual faltan comas ({error})"
57
58     # ordena al bot enviar un mensaje de vuelta al chat de telegram
59     await update.message.reply_text(respuesta)
60
61 # arrancar Bot, usa el token para conectarse al servidor de telegram
62 app = ApplicationBuilder().token(TOKEN).build()
63 # tiene que estar atento a los mensajes y que solo quiere texto y llamamos a la
64 # funcion
65 app.add_handler(MessageHandler(filters.TEXT, manejar_mensaje))
66 print("Bot funcionando...")
67 # el bot se queda escuchando constantemente
68 app.run_polling()

```

Para comprobar que funciona correctamente, abrimos la terminal y ejecutamos: "python bot socios.py". Si funciona nos sale lo siguiente:

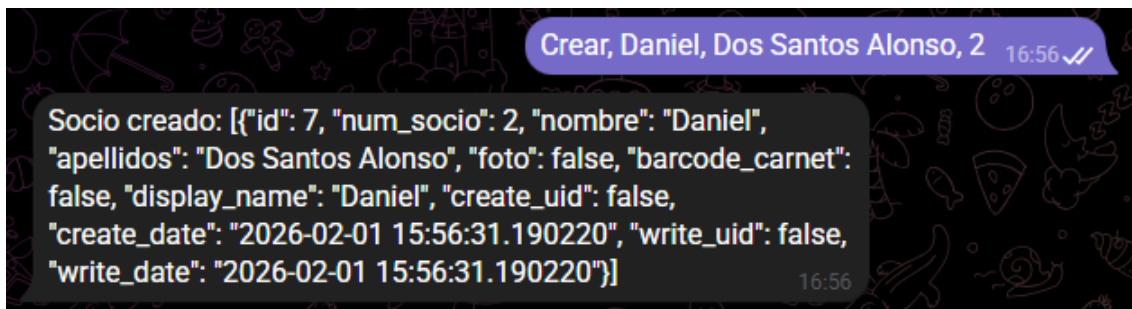


```

PS C:\Users\andre\Desktop\2CS_DAM\SGestionEmpresarial\Practica2_Implementacion_Odoo> python bot_socios.py
Bot funcionando...

```

Ahora vamos a telegram, al bot creado y vamos a probar a crear un nuevo socio. Insertamos los datos y nos devuelve un mensaje:

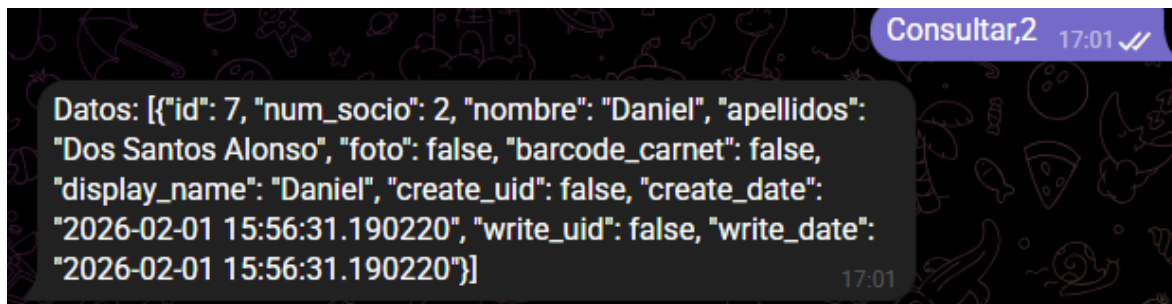


Y comprobamos que en Odoo actualizando que se ha creado correctamente.

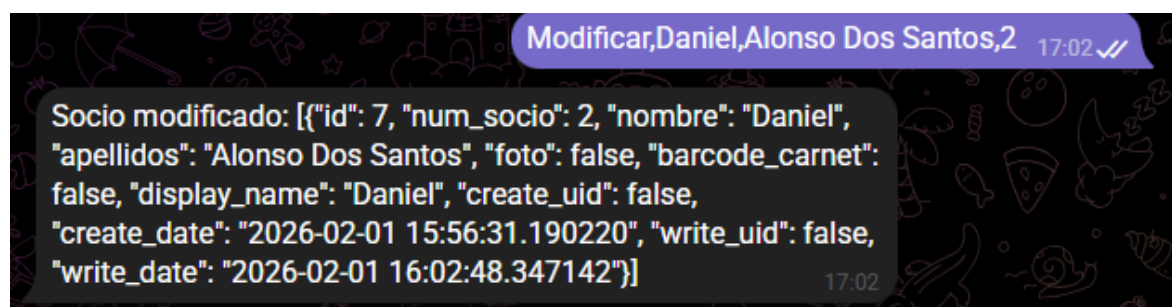
Gestión de socios Socios		
Nuevo Listado de socios ⚙️		
1-2 / 2 < > [Menu Icon] [Print Icon]		
🔍 Buscar...		
☐ Número ...	Nombre	Apellidos
☐ 1	Andrea Sofia	Pais Dos Santos
☐ 2	Daniel	Dos Santos Alonso

Vemos que esta operación funciona correctamente, vamos a comprobar que las otras 2 funcionan bien:

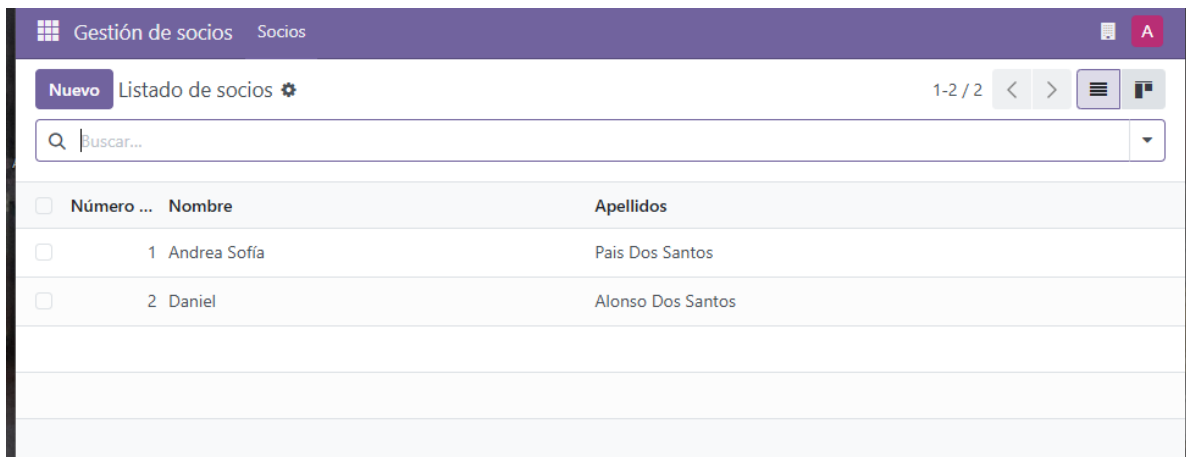
- Consultar: a través del número de socio podemos ver los datos del socio que hemos creado anteriormente.



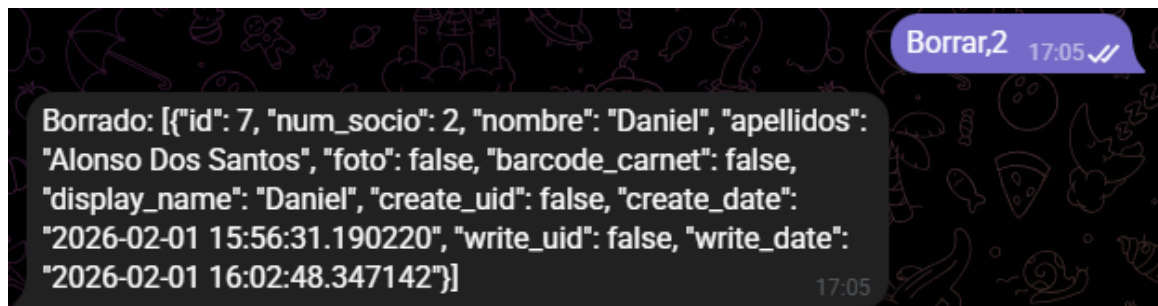
- Modificar: vamos a cambiar los apellidos de orden:



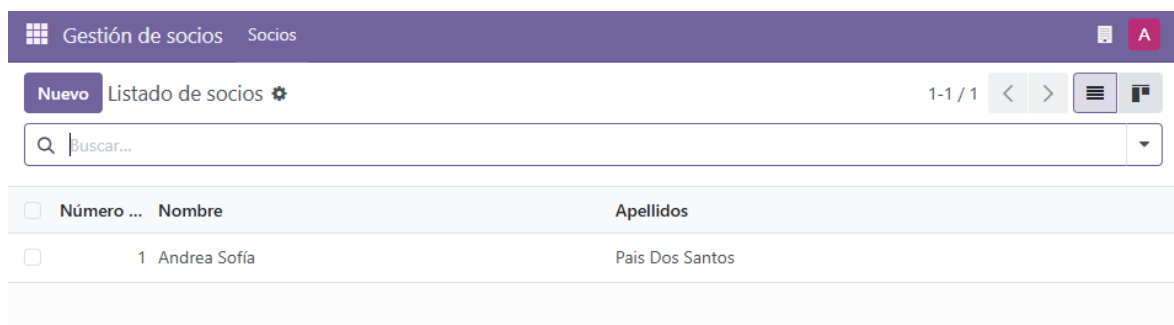
Actualizamos en Odoo y vemos que los apellidos se han cambiado.



- Borrar: con la orden y el número de socio podemos borrar un socio, y he borrado el que he creado antes.



Actualizamos Odoo y vemos que ya no existe ese socio.



Sí le decimos al bot cualquiera cosa que no sea una de esas órdenes nos devuelve este mensaje:



6 Actividad 04 - Generar de imágenes aleatorias con Web Controller

Para esta actividad nos vamos a basar en el módulo 9 del repositorio aportado. Con esa base primero vamos a instalar la librería Pillow en el entorno de Python usando el comando "python -m pip install Pillow".

```
PS C:\Users\andre\Desktop\2CS_DAM\SGestionEmpresarial\Practica2_Implementacion_Odoo> python -m pip install Pillow
Collecting Pillow
  Downloading pillow-12.1.0-cp314-cp314-win_amd64.whl.metadata (9.0 kB)
  Downloading pillow-12.1.0-cp314-cp314-win_amd64.whl (7.2 MB)
    7.2/7.2 MB 31.9 MB/s 0:00:00
Installing collected packages: Pillow
Successfully installed Pillow-12.1.0

[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: C:\Users\andre\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip
```

Ahora creamos en la carpeta "controllers" y nuevo archivo "controlador" que se encargará de generar la imagen utiliza la librería de procesamiento de imágenes Pillow para crear una representación visual de datos aleatorios.

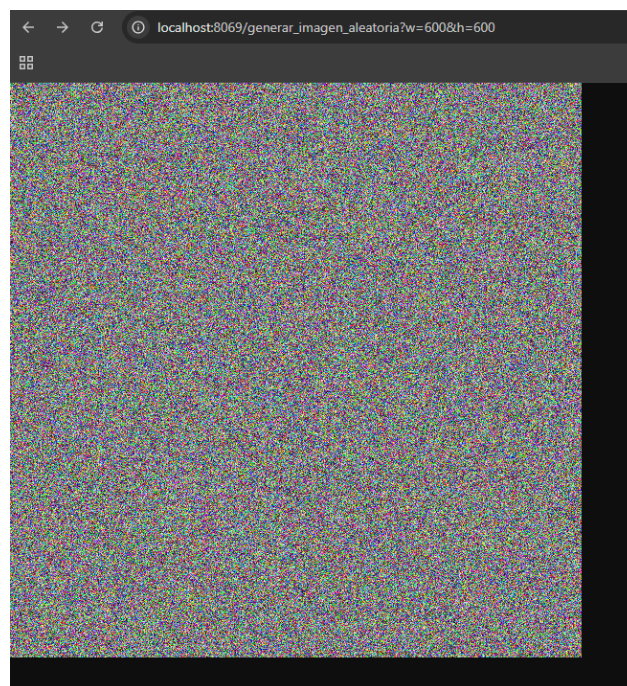
```
1  # -*- coding: utf-8 -*-
2  from odoo import http
3  from odoo.http import request
4  import io
5  import random
6
7  # Intentar importar Pillow de forma segura
8  try:
9      from PIL import Image
10 except ImportError:
11     Image = None
12
13 class ImagenAleatoriaController(http.Controller):
14
15     @http.route('/generar_imagen_aleatoria', type='http', auth='public', website=
16         True)
17     def crear_imagen(self, w=200, h=200, **kwargs):
18         # Si Pillow no esta, se devuelve texto para no dar Error 500
19         if Image is None:
20             return "Error: La libreria Pillow no esta instalada en el contenedor
21                 Odoo."
22
23         try:
24             ancho = int(w)
25             alto = int(h)
26         except:
27             ancho, alto = 200, 200
28
29         # Crear la imagen aleatoria
30         img = Image.new('RGB', (ancho, alto))
31         pixeles = img.load()
32         for x in range(ancho):
33             for y in range(alto):
34                 pixeles[x, y] = (random.randint(0, 255), random.randint(0, 255),
35                                 random.randint(0, 255))
```

```

34     # Guardar imagen en buffer
35     output = io.BytesIO()
36     img.save(output, format="PNG")
37     contenido = output.getvalue()
38     output.close()
39
40     # Retornar respuesta simple para evitar descargas
41     return request.make_response(contenido, [
42         ('Content-Type', 'image/png'),
43         ('Content-Disposition', 'inline')
44     ])

```

En el archivo "init" del módulo hay que importar el controler y para probarlo vamos a actualizar el módulo primero y luego con la url accedemos al generador de imagen: [generar imagen](#).



7 Conclusiones

Al empezar la práctica parecía bastante compleja, pedía cosas completamente diferente a lo que estábamos acostumbrados, al ir haciendo las actividades vi que la primera fue la que más tardé entonces las demás se me hicieron más amenas. Fue interesante ver como se configuraba el Bot de Telegram para que comunique con Odoo, era algo confuso el tema de como tratar los datos, pero como se trataba de las operaciones básicas era fácil de entender.

Tuve suerte de que no tuve muchos errores, y la mayoría eran por ir a prisas escribiendo y confundía los nombres de los archivos.

En conclusión, fue un trabajo bastante largo pero la verdad es que aprendí bastante en poco tiempo como estructurar los módulos correctamente e ir comprobando que todo funcione correctamente.